

## ✓ WattwAIs Deep Learning Regression Models

This notebook trains deep learning models to predict hourly household electricity demand.

Target variable:

Demand\_kWh

The dataset is treated as tabular regression data, not pure sequential time-series data.

Models used:

- MLP
- Residual MLP
- Wide & Deep Neural Network
- 1D CNN
- GRU

The final output includes:

- Model name
- MAE
- RMSE
- $R^2$
- MAPE
- Actual vs predicted sample
- Best model saved as .keras

## ✓ 1. Upload Cleaned Dataset

Upload the cleaned WattwAIs dataset CSV file.

```
from google.colab import files
```

```
uploaded = files.upload()
```

wattwais\_ifl...dataset.csv

**wattwais\_iflex\_cleaned\_dataset.csv**(text/csv) - 27511234 bytes, last modified: 5/21/2026 - 100% done  
Saving wattwais\_iflex\_cleaned\_dataset.csv to wattwais\_iflex\_cleaned\_dataset.csv

## ✓ 2. Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import json
import os
```

```

import shutil
import joblib

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input,
    Dense,
    Dropout,
    BatchNormalization,
    Add,
    Concatenate,
    Conv1D,
    Flatten,
    GRU
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

print("tensorflow version:", tf.__version__)

```

tensorflow version: 2.21.0

### ✓ 3. Load Dataset

```

filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("loaded file:", filename)
print("dataset shape:", df.shape)

df.head()

```

loaded file: wattwais\_iflex\_cleaned\_dataset.csv  
dataset shape: (170088, 24)

|   | ID      | From                         | Date       | Demand_kWh | hour | day_of_week | month | day | is_weekend | E |
|---|---------|------------------------------|------------|------------|------|-------------|-------|-----|------------|---|
| 0 | Exp_135 | 2020-02-11<br>00:00:00+00:00 | 11/02/2020 | 0.408      | 0    | 1           | 2     | 11  | 0          |   |
| 1 | Exp_35  | 2020-02-11<br>00:00:00+00:00 | 11/02/2020 | 1.731      | 0    | 1           | 2     | 11  | 0          |   |
| 2 | Exp_159 | 2020-02-11<br>00:00:00+00:00 | 11/02/2020 | 1.127      | 0    | 1           | 2     | 11  | 0          |   |
| 3 | Exp_191 | 2020-02-11<br>00:00:00+00:00 | 11/02/2020 | 0.793      | 0    | 1           | 2     | 11  | 0          |   |
| 4 | Exp_44  | 2020-02-11<br>00:00:00+00:00 | 11/02/2020 | 1.064      | 0    | 1           | 2     | 11  | 0          |   |

5 rows × 24 columns

## ✓ 4. Inspect Dataset

```
print("columns:")
print(df.columns.tolist())

print("\ndataset info:")
df.info()

df.describe(include="all").T
```



```
dataset info:  
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 170088 entries, 0 to 170087  
Data columns (total 24 columns):
```

The target variable is Demand\_kwh, which represents hourly electricity demand.

```
df = df.copy()
```

```
df["From"] = pd.to_datetime(df["From"], errors="coerce")
df = df.sort_values("From").reset_index(drop=True)
```

```
print("cleaned dataset shape:", df.shape)
df.head()
```

```
cleaned dataset shape: (170088, 24)
```

| cleaned dataset shape: (170088, 24) |               |                           |            |            |            |             |           |          |            |   |
|-------------------------------------|---------------|---------------------------|------------|------------|------------|-------------|-----------|----------|------------|---|
|                                     | Date          |                           | 170088.0   | 55         | 18/01/2021 | 7728        | NaN       | NaN      |            |   |
|                                     | ID            | From                      | Date       | Demand_kwh | hour       | day_of_week | month     | day      | is_weekend | E |
|                                     | Demand_kWh    |                           | 170088.0   | NaN        | NaN        | NaN         | 2.949261  | 2.216123 |            |   |
| 0                                   | Exp_135       | 2020-02-11 00:00:00+00:00 | 11/02/2020 | 0.408      | 0          | 1           | 2         | 11       |            | 0 |
| 1                                   | Exp_61        | 2020-02-11 00:00:00+00:00 | 170088.0   | NaN        | NaN        | NaN         | 1.839001  | 1.407983 |            | 0 |
|                                     |               |                           | 11/02/2020 | 0.430      | 0          | 1           | 2         | 11       |            |   |
|                                     |               |                           | 170088.0   | NaN        | NaN        | NaN         | 2.567095  | 2.690312 |            |   |
| 2                                   | Exp_208       | 2020-02-11 00:00:00+00:00 | 11/02/2020 | 2.692      | 0          | 1           | 2         | 11       |            | 0 |
| 3                                   | Exp_62        | 2020-02-11 00:00:00+00:00 | 170088.0   | NaN        | NaN        | NaN         | 0.0       | 0.0      |            | 0 |
|                                     |               |                           | 11/02/2020 | 1.538      | 0          | 1           | 2         | 11       |            |   |
|                                     |               |                           | 170088.0   | NaN        | NaN        | NaN         | 3.374264  | 4.260177 |            |   |
| 4                                   | Exp_75        | 2020-02-11 00:00:00+00:00 | 11/02/2020 | 5.690      | 0          | 1           | 2         | 11       |            | 0 |
| 5 rows × 24 columns                 |               |                           |            |            |            |             |           |          |            |   |
|                                     | Temperature24 |                           | 170088.0   | NaN        | NaN        | NaN         | -3.515884 | 4.737956 |            | - |
|                                     | Temperature48 |                           | 170088.0   | NaN        | NaN        | NaN         | -3.478987 | 4.763963 |            | - |
|                                     | Temperature72 |                           | 170088.0   | NaN        | NaN        | NaN         | -3.427198 | 4.69294  |            | - |

## 6. Define Features and Target

|             |          |     |     |     |          |          |
|-------------|----------|-----|-----|-----|----------|----------|
| temperature | 170088.0 | NaN | NaN | NaN | 2.949267 | 2.215333 |
| lag_24      | 170088.0 | NaN | NaN | NaN | 2.965534 | 2.228547 |
| lag_100     | 170088.0 | NaN | NaN | NaN | 2.80291  | 2.113543 |
| rolling_24  | 170088.0 | NaN | NaN | NaN | 2.966817 | 2.001559 |
| rolling_100 | 170088.0 | NaN | NaN | NaN | 2.966817 | 2.001559 |

The model predicts Demand\_kWh using available time, weather, pricing, household, and historical consumption features.

```
excluded_cols = [
    target_col,
    "From",
    "Date"
]

feature_cols = [col for col in df.columns if col not in excluded_cols]

numeric_features = df[feature_cols].select_dtypes(
    include=["int64", "float64", "int32", "float32"]
).columns.tolist()

categorical_features = df[feature_cols].select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()

print("target column:", target_col)
print("\nnumeric features:")
print(numeric_features)

print("\ncategorical features:")
print(categorical_features)

print("\nnumber of input features:", len(feature_cols))
```

target column: Demand\_kWh

numeric features:

['hour', 'day\_of\_week', 'month', 'day', 'is\_weekend', 'Experiment\_price\_NOK\_kWh', 'Temperature', 'lag\_24', 'lag\_100', 'rolling\_24', 'rolling\_100']

categorical features:

['ID', 'Region', 'Municipality', 'Participation\_Phase', 'Control\_Price\_Phase2', 'Group\_Phase2']

number of input features: 21

## 7. Handle Missing Values

```

df_model = df.copy()

for col in numeric_features:
    df_model[col] = df_model[col].fillna(df_model[col].median())

for col in categorical_features:
    df_model[col] = df_model[col].fillna("Unknown").astype(str)

print("remaining missing values:")
print(df_model[feature_cols + [target_col]].isnull().sum().sort_values(ascending=False).head

```

```

remaining missing values:
ID                                0
hour                              0
day_of_week                       0
month                             0
day                               0
is_weekend                        0
Experiment_price_NOK_kWh          0
Temperature                       0
Temperature24                     0
Temperature48                     0
Temperature72                     0
lag_1                             0
lag_24                            0
lag_168                           0
rolling_24                        0
rolling_168                       0
Region                            0
Municipality                      0
Participation_Phase               0
Control_Price_Phase2              0
dtype: int64

```

## ✓ 8. Train-Test Split

Since the data contains time-based electricity records, the split is done without shuffling. The earlier 80% of the data is used for training, while the later 20% is used for testing.

```

if "From" in df_model.columns:
    df_model = df_model.sort_values("From").reset_index(drop=True)

split_index = int(len(df_model) * 0.8)

train_df = df_model.iloc[:split_index].copy()
test_df = df_model.iloc[split_index:].copy()

X_train = train_df[feature_cols]
y_train = train_df[target_col].values

X_test = test_df[feature_cols]
y_test = test_df[target_col].values

print("training shape:", X_train.shape)
print("testing shape:", X_test.shape)

```

```
training shape: (136070, 21)
testing shape: (34018, 21)
```

## ✓ 9. Preprocess Features

Numeric features are standardized, while categorical features are one-hot encoded.

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False), categorical_fea
    ],
    remainder="drop"
)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

X_train_processed = np.asarray(X_train_processed).astype("float32")
X_test_processed = np.asarray(X_test_processed).astype("float32")

y_train = np.asarray(y_train).astype("float32")
y_test = np.asarray(y_test).astype("float32")

input_dim = X_train_processed.shape[1]

print("processed training shape:", X_train_processed.shape)
print("processed testing shape:", X_test_processed.shape)
print("input dimension:", input_dim)

processed training shape: (136070, 399)
processed testing shape: (34018, 399)
input dimension: 399
```

## ✓ 10. Evaluation Functions

```
def safe_mape(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)

    mask = y_true != 0

    if mask.sum() == 0:
        return np.nan

    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

def evaluate_model(model_name, model, X_eval, y_true):
    y_pred = model.predict(X_eval, verbose=0).flatten()

    mae = mean_absolute_error(y_true, y_pred)
```

```

rmse = np.sqrt(mean_squared_error(y_true, y_pred))
r2 = r2_score(y_true, y_pred)
mape = safe_mape(y_true, y_pred)

result = {
    "Model": model_name,
    "MAE": mae,
    "RMSE": rmse,
    "R2": r2,
    "MAPE": mape
}

return result, y_pred

```

## ✓ 11. Training Callback

Early stopping prevents overfitting by stopping training when validation loss no longer improves.

```

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

```

## ✓ 12. MLP Model

```

def build_mlp(input_dim):
    inputs = Input(shape=(input_dim,))

    x = Dense(128, activation="relu")(inputs)
    x = BatchNormalization()(x)
    x = Dropout(0.2)(x)

    x = Dense(64, activation="relu")(x)
    x = BatchNormalization()(x)
    x = Dropout(0.2)(x)

    x = Dense(32, activation="relu")(x)

    outputs = Dense(1)(x)

    model = Model(inputs, outputs)

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae"]
    )

    return model

```

```
mlp_model = build_mlp(input_dim)
mlp_model.summary()
```

Model: "functional"

| Layer (type)                                                 | Output Shape                  | Param # |
|--------------------------------------------------------------|-------------------------------|---------|
| input_layer ( <a href="#">InputLayer</a> )                   | ( <a href="#">None</a> , 399) | 0       |
| dense ( <a href="#">Dense</a> )                              | ( <a href="#">None</a> , 128) | 51,200  |
| batch_normalization ( <a href="#">BatchNormalization</a> )   | ( <a href="#">None</a> , 128) | 512     |
| dropout ( <a href="#">Dropout</a> )                          | ( <a href="#">None</a> , 128) | 0       |
| dense_1 ( <a href="#">Dense</a> )                            | ( <a href="#">None</a> , 64)  | 8,256   |
| batch_normalization_1 ( <a href="#">BatchNormalization</a> ) | ( <a href="#">None</a> , 64)  | 256     |
| dropout_1 ( <a href="#">Dropout</a> )                        | ( <a href="#">None</a> , 64)  | 0       |
| dense_2 ( <a href="#">Dense</a> )                            | ( <a href="#">None</a> , 32)  | 2,080   |
| dense_3 ( <a href="#">Dense</a> )                            | ( <a href="#">None</a> , 1)   | 33      |

Total params: 62,337 (243.50 KB)  
Total size of the model is 243.50 KB

```
mlp_history = mlp_model.fit(
    X_train_processed,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=[early_stop],
    verbose=1
)
```

```

Epoch 26/100
426/426 ————— 2s 5ms/step - loss: 0.5914 - mae: 0.5162 - val_loss: 0.5794 - \
Epoch 27/100
426/426 ————— 3s 7ms/step - loss: 0.5859 - mae: 0.5140 - val_loss: 0.5849 - \
Epoch 28/100
426/426 ————— 4s 4ms/step - loss: 0.5861 - mae: 0.5135 - val_loss: 0.6366 - \
Epoch 29/100
426/426 ————— 2s 5ms/step - loss: 0.5820 - mae: 0.5112 - val_loss: 0.5855 - \
Epoch 30/100
426/426 ————— 2s 5ms/step - loss: 0.5823 - mae: 0.5116 - val_loss: 0.5839 - \
Epoch 31/100
426/426 ————— 2s 5ms/step - loss: 0.5815 - mae: 0.5111 - val_loss: 0.6407 - \
Epoch 32/100
426/426 ————— 2s 5ms/step - loss: 0.5766 - mae: 0.5092 - val_loss: 0.6066 - \
Epoch 33/100
426/426 ————— 3s 6ms/step - loss: 0.5758 - mae: 0.5081 - val_loss: 0.5709 - \
Epoch 34/100
426/426 ————— 2s 4ms/step - loss: 0.5707 - mae: 0.5057 - val_loss: 0.5963 - \
Epoch 35/100
426/426 ————— 3s 8ms/step - loss: 0.5713 - mae: 0.5064 - val_loss: 0.6355 - \
Epoch 36/100
426/426 ————— 5s 12ms/step - loss: 0.5690 - mae: 0.5049 - val_loss: 0.6342 - \
Epoch 37/100
426/426 ————— 7s 5ms/step - loss: 0.5689 - mae: 0.5050 - val_loss: 0.5820 - \
Epoch 38/100
426/426 ————— 2s 5ms/step - loss: 0.5635 - mae: 0.5040 - val_loss: 0.5955 - \
Epoch 39/100
426/426 ————— 3s 7ms/step - loss: 0.5621 - mae: 0.5030 - val_loss: 0.5849 - \
Epoch 40/100
426/426 ————— 5s 5ms/step - loss: 0.5603 - mae: 0.5017 - val_loss: 0.5895 - \
Epoch 41/100
426/426 ————— 2s 5ms/step - loss: 0.5572 - mae: 0.4996 - val_loss: 0.5960 - \
Epoch 42/100
426/426 ————— 2s 4ms/step - loss: 0.5561 - mae: 0.5001 - val_loss: 0.5876 - \
Epoch 43/100
426/426 ————— 2s 4ms/step - loss: 0.5540 - mae: 0.4992 - val_loss: 0.5886 - \

```

## ✓ 13. Residual MLP Model

```

def residual_block(x, units):
    shortcut = Dense(units)(x)

    x = Dense(units, activation="relu")(x)
    x = BatchNormalization()(x)
    x = Dropout(0.2)(x)

    x = Dense(units)(x)
    x = BatchNormalization()(x)

    x = Add()(x, shortcut)
    x = tf.keras.layers.Activation("relu")(x)

    return x

def build_residual_mlp(input_dim):
    inputs = Input(shape=(input_dim,))

```

```
x = Dense(128, activation="relu")(inputs)
x = residual_block(x, 128)
x = residual_block(x, 64)

x = Dense(32, activation="relu")(x)

outputs = Dense(1)(x)

model = Model(inputs, outputs)

model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="mse",
    metrics=["mae"]
)

return model

residual_mlp_model = build_residual_mlp(input_dim)
residual_mlp_model.summary()
```

Model: "functional\_1"

| Layer (type)                                   | Output Shape | Param # | Connected to                         |
|------------------------------------------------|--------------|---------|--------------------------------------|
| input_layer_1<br>(InputLayer)                  | (None, 399)  | 0       | -                                    |
| dense_4 (Dense)                                | (None, 128)  | 51,200  | input_layer_1[0]...                  |
| dense_6 (Dense)                                | (None, 128)  | 16,512  | dense_4[0][0]                        |
| batch_normalizatio...<br>(BatchNormalizatio... | (None, 128)  | 512     | dense_6[0][0]                        |
| dropout_2 (Dropout)                            | (None, 128)  | 0       | batch_normalizat...                  |
| dense_7 (Dense)                                | (None, 128)  | 16,512  | dropout_2[0][0]                      |
| batch_normalizatio...<br>(BatchNormalizatio... | (None, 128)  | 512     | dense_7[0][0]                        |
| dense_5 (Dense)                                | (None, 128)  | 16,512  | dense_4[0][0]                        |
| add (Add)                                      | (None, 128)  | 0       | batch_normalizat...<br>dense_5[0][0] |
| activation<br>(Activation)                     | (None, 128)  | 0       | add[0][0]                            |
| dense_9 (Dense)                                | (None, 64)   | 8,256   | activation[0][0]                     |
| batch_normalizatio...<br>(BatchNormalizatio... | (None, 64)   | 256     | dense_9[0][0]                        |
| dropout_3 (Dropout)                            | (None, 64)   | 0       | batch_normalizat...                  |
| dense_10 (Dense)                               | (None, 64)   | 4,160   | dropout_3[0][0]                      |
| batch_normalizatio...<br>(BatchNormalizatio... | (None, 64)   | 256     | dense_10[0][0]                       |
| dense_8 (Dense)                                | (None, 64)   | 8,256   | activation[0][0]                     |
| add_1 (Add)                                    | (None, 64)   | 0       | batch_normalizat...<br>dense_8[0][0] |
| activation_1<br>(Activation)                   | (None, 64)   | 0       | add_1[0][0]                          |
| dense_11 (Dense)                               | (None, 32)   | 2,080   | activation_1[0][...                  |
| dense_12 (Dense)                               | (None, 1)    | 33      | dense_11[0][0]                       |

```
residual_mlp_history = residual_mlp_model.fit(
    X_train_processed,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=[early_stop],
    verbose=1
```

)

Epoch 1/100

426/426  15s 16ms/step - loss: 0.8816 - mae: 0.6450 - val\_loss: 0.7090 - va

Epoch 2/100

426/426  2s 5ms/step - loss: 0.6711 - mae: 0.5576 - val\_loss: 0.5997 - va

Epoch 3/100

426/426  2s 4ms/step - loss: 0.6405 - mae: 0.5388 - val\_loss: 0.5868 - va

Epoch 4/100

426/426  2s 4ms/step - loss: 0.6138 - mae: 0.5261 - val\_loss: 0.5939 - va

Epoch 5/100

426/426  2s 4ms/step - loss: 0.6060 - mae: 0.5215 - val\_loss: 0.5882 - va

Epoch 6/100

426/426  2s 4ms/step - loss: 0.5930 - mae: 0.5142 - val\_loss: 0.5838 - va

Epoch 7/100

426/426  2s 4ms/step - loss: 0.5803 - mae: 0.5083 - val\_loss: 0.5799 - va

Epoch 8/100

426/426  2s 6ms/step - loss: 0.5752 - mae: 0.5055 - val\_loss: 0.6045 - va

Epoch 9/100

426/426  2s 4ms/step - loss: 0.5678 - mae: 0.5015 - val\_loss: 0.5766 - va

Epoch 10/100

426/426  2s 4ms/step - loss: 0.5573 - mae: 0.4969 - val\_loss: 0.5892 - va

## ✓ 14. Wide & Deep Model

```
def build_wide_deep(input_dim):
    inputs = Input(shape=(input_dim,))

    wide = Dense(64, activation="linear")(inputs)

    deep = Dense(128, activation="relu")(inputs)
    deep = BatchNormalization()(deep)
    deep = Dropout(0.2)(deep)

    deep = Dense(64, activation="relu")(deep)
    deep = BatchNormalization()(deep)
    deep = Dropout(0.2)(deep)

    deep = Dense(32, activation="relu")(deep)

    combined = Concatenate()([wide, deep])

    x = Dense(64, activation="relu")(combined)

    outputs = Dense(1)(x)

    model = Model(inputs, outputs)

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae"]
    )

    return model
```

```
wide_deep_model = build_wide_deep(input_dim)
wide_deep_model.summary()
```

Model: "functional\_2"

| Layer (type)                                   | Output Shape | Param # | Connected to                      |
|------------------------------------------------|--------------|---------|-----------------------------------|
| input_layer_2<br>(InputLayer)                  | (None, 399)  | 0       | -                                 |
| dense_14 (Dense)                               | (None, 128)  | 51,200  | input_layer_2[0]...               |
| batch_normalizatio...<br>(BatchNormalizatio... | (None, 128)  | 512     | dense_14[0][0]                    |
| dropout_4 (Dropout)                            | (None, 128)  | 0       | batch_normalizat...               |
| dense_15 (Dense)                               | (None, 64)   | 8,256   | dropout_4[0][0]                   |
| batch_normalizatio...<br>(BatchNormalizatio... | (None, 64)   | 256     | dense_15[0][0]                    |
| dropout_5 (Dropout)                            | (None, 64)   | 0       | batch_normalizat...               |
| dense_13 (Dense)                               | (None, 64)   | 25,600  | input_layer_2[0]...               |
| dense_16 (Dense)                               | (None, 32)   | 2,080   | dropout_5[0][0]                   |
| concatenate<br>(Concatenate)                   | (None, 96)   | 0       | dense_13[0][0],<br>dense_16[0][0] |
| dense_17 (Dense)                               | (None, 64)   | 6,208   | concatenate[0][0]                 |
| dense_18 (Dense)                               | (None, 1)    | 65      | dense_17[0][0]                    |

Total params: 94,177 (367.88 KB)

```
wide_deep_history = wide_deep_model.fit(
    X_train_processed,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=[early_stop],
    verbose=1
)
```

Epoch 1/100

426/426  11s 14ms/step - loss: 0.9203 - mae: 0.6260 - val\_loss: 0.6010 -

Epoch 2/100

426/426  2s 4ms/step - loss: 0.6392 - mae: 0.5362 - val\_loss: 0.6032 - va

Epoch 3/100

426/426  2s 4ms/step - loss: 0.6297 - mae: 0.5306 - val\_loss: 0.5885 - va

Epoch 4/100

426/426  3s 8ms/step - loss: 0.6209 - mae: 0.5265 - val\_loss: 0.5910 - va

Epoch 5/100

426/426  5s 8ms/step - loss: 0.6110 - mae: 0.5213 - val\_loss: 0.5793 - va

Epoch 6/100

426/426  3s 4ms/step - loss: 0.6016 - mae: 0.5170 - val\_loss: 0.5668 - va

Epoch 7/100

```

426/426 ————— 2s 4ms/step - loss: 0.5974 - mae: 0.5155 - val_loss: 0.5755 - va
Epoch 8/100
426/426 ————— 2s 4ms/step - loss: 0.5888 - mae: 0.5108 - val_loss: 0.5715 - va
Epoch 9/100
426/426 ————— 2s 4ms/step - loss: 0.5816 - mae: 0.5080 - val_loss: 0.5664 - va
Epoch 10/100
426/426 ————— 2s 4ms/step - loss: 0.5743 - mae: 0.5048 - val_loss: 0.5638 - va
Epoch 11/100
426/426 ————— 2s 5ms/step - loss: 0.5687 - mae: 0.5022 - val_loss: 0.5583 - va
Epoch 12/100
426/426 ————— 2s 4ms/step - loss: 0.5653 - mae: 0.5003 - val_loss: 0.5627 - va
Epoch 13/100
426/426 ————— 2s 4ms/step - loss: 0.5601 - mae: 0.4978 - val_loss: 0.5696 - va
Epoch 14/100
426/426 ————— 2s 4ms/step - loss: 0.5553 - mae: 0.4951 - val_loss: 0.5584 - va
Epoch 15/100
426/426 ————— 2s 4ms/step - loss: 0.5491 - mae: 0.4922 - val_loss: 0.5592 - va
Epoch 16/100
426/426 ————— 2s 4ms/step - loss: 0.5462 - mae: 0.4914 - val_loss: 0.5672 - va
Epoch 17/100
426/426 ————— 2s 4ms/step - loss: 0.5419 - mae: 0.4889 - val_loss: 0.5663 - va
Epoch 18/100
426/426 ————— 2s 5ms/step - loss: 0.5389 - mae: 0.4871 - val_loss: 0.5624 - va
Epoch 19/100
426/426 ————— 2s 4ms/step - loss: 0.5349 - mae: 0.4849 - val_loss: 0.5623 - va
Epoch 20/100
426/426 ————— 2s 4ms/step - loss: 0.5303 - mae: 0.4833 - val_loss: 0.5635 - va
Epoch 21/100
426/426 ————— 2s 4ms/step - loss: 0.5288 - mae: 0.4832 - val_loss: 0.5661 - va

```

## ✓ 15. Prepare Reshaped Input for 1D CNN and GRU

The 1D CNN and GRU models require 3D input.

Since this dataset is tabular, the processed feature vector is reshaped into a feature sequence representation.

```

X_train_seq = X_train_processed.reshape(
    X_train_processed.shape[0],
    X_train_processed.shape[1],
    1
)

X_test_seq = X_test_processed.reshape(
    X_test_processed.shape[0],
    X_test_processed.shape[1],
    1
)

print("sequential training shape:", X_train_seq.shape)
print("sequential testing shape:", X_test_seq.shape)

sequential training shape: (136070, 399, 1)
sequential testing shape: (34018, 399, 1)

```

## 16. 1D CNN Model

```
def build_1d_cnn(input_dim):
    inputs = Input(shape=(input_dim, 1))

    x = Conv1D(filters=64, kernel_size=3, activation="relu", padding="same")(inputs)
    x = BatchNormalization()(x)

    x = Conv1D(filters=32, kernel_size=3, activation="relu", padding="same")(x)
    x = BatchNormalization()(x)

    x = Flatten()(x)

    x = Dense(64, activation="relu")(x)
    x = Dropout(0.2)(x)

    outputs = Dense(1)(x)

    model = Model(inputs, outputs)

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae"]
    )

    return model

cnn_model = build_1d_cnn(input_dim)
cnn_model.summary()
```

Model: "functional\_3"

| Layer (type)                               | Output Shape    | Param # |
|--------------------------------------------|-----------------|---------|
| input_layer_3 (InputLayer)                 | (None, 399, 1)  | 0       |
| conv1d (Conv1D)                            | (None, 399, 64) | 256     |
| batch_normalization_8 (BatchNormalization) | (None, 399, 64) | 256     |
| conv1d_1 (Conv1D)                          | (None, 399, 32) | 6,176   |
| batch_normalization_9 (BatchNormalization) | (None, 399, 32) | 128     |
| flatten (Flatten)                          | (None, 12768)   | 0       |
| dense_19 (Dense)                           | (None, 64)      | 817,216 |
| dropout_6 (Dropout)                        | (None, 64)      | 0       |
| dense_20 (Dense)                           | (None, 1)       | 65      |

Total params: 824,097 (3.14 MB)

```

cnn_history = cnn_model.fit(
    X_train_seq,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=[early_stop],
    verbose=1
)

```

```

Epoch 1/100
426/426 ————— 23s 34ms/step - loss: 14.3454 - mae: 2.9388 - val_loss: 12.8301
Epoch 2/100
426/426 ————— 28s 18ms/step - loss: 11.5968 - mae: 2.5845 - val_loss: 10.8731
Epoch 3/100
426/426 ————— 7s 16ms/step - loss: 9.8807 - mae: 2.3356 - val_loss: 9.2936 - v
Epoch 4/100
426/426 ————— 7s 16ms/step - loss: 8.5189 - mae: 2.1454 - val_loss: 8.0507 - v
Epoch 5/100
426/426 ————— 9s 21ms/step - loss: 7.4686 - mae: 2.0027 - val_loss: 7.1003 - v
Epoch 6/100
426/426 ————— 8s 19ms/step - loss: 6.6838 - mae: 1.9020 - val_loss: 6.3960 - v
Epoch 7/100
426/426 ————— 9s 17ms/step - loss: 6.1206 - mae: 1.8382 - val_loss: 5.8977 - v
Epoch 8/100
426/426 ————— 7s 16ms/step - loss: 5.7403 - mae: 1.8018 - val_loss: 5.5655 - v
Epoch 9/100
426/426 ————— 7s 17ms/step - loss: 5.5010 - mae: 1.7860 - val_loss: 5.3598 - v
Epoch 10/100
426/426 ————— 7s 17ms/step - loss: 5.3648 - mae: 1.7833 - val_loss: 5.2432 - v

```

## ✓ 17. GRU Model

```

def build_gru(input_dim):
    inputs = Input(shape=(input_dim, 1))

    x = GRU(64, return_sequences=False)(inputs)
    x = Dropout(0.2)(x)

    x = Dense(64, activation="relu")(x)
    x = BatchNormalization()(x)

    x = Dense(32, activation="relu")(x)

    outputs = Dense(1)(x)

    model = Model(inputs, outputs)

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss="mse",
        metrics=["mae"]
    )

    return model

```

```
gru_model = build_gru(input_dim)
gru_model.summary()
```

Model: "functional\_4"

| Layer (type)                                | Output Shape   | Param # |
|---------------------------------------------|----------------|---------|
| input_layer_4 (InputLayer)                  | (None, 399, 1) | 0       |
| gru (GRU)                                   | (None, 64)     | 12,864  |
| dropout_7 (Dropout)                         | (None, 64)     | 0       |
| dense_21 (Dense)                            | (None, 64)     | 4,160   |
| batch_normalization_10 (BatchNormalization) | (None, 64)     | 256     |
| dense_22 (Dense)                            | (None, 32)     | 2,080   |
| dense_23 (Dense)                            | (None, 1)      | 33      |

Total params: 19,393 (75.75 KB)

Trainable params: 19,365 (75.75 KB)

```
gru_history = gru_model.fit(
    X_train_seq,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=256,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/100
426/426 ————— 16s 23ms/step - loss: 5.6384 - mae: 1.8559 - val_loss: 7.8007 -
Epoch 2/100
426/426 ————— 10s 24ms/step - loss: 5.1481 - mae: 1.7796 - val_loss: 5.2547 -
Epoch 3/100
426/426 ————— 10s 24ms/step - loss: 5.0155 - mae: 1.7510 - val_loss: 4.8971 -
Epoch 4/100
426/426 ————— 10s 24ms/step - loss: 4.9885 - mae: 1.7463 - val_loss: 4.8688 -
Epoch 5/100
426/426 ————— 10s 23ms/step - loss: 4.9497 - mae: 1.7383 - val_loss: 4.9701 -
Epoch 6/100
426/426 ————— 9s 22ms/step - loss: 4.9104 - mae: 1.7327 - val_loss: 4.7827 - v
Epoch 7/100
426/426 ————— 10s 23ms/step - loss: 4.8548 - mae: 1.7202 - val_loss: 4.8953 -
Epoch 8/100
426/426 ————— 10s 23ms/step - loss: 4.8019 - mae: 1.7072 - val_loss: 4.7592 -
Epoch 9/100
426/426 ————— 10s 23ms/step - loss: 4.7613 - mae: 1.6979 - val_loss: 4.9119 -
Epoch 10/100
426/426 ————— 10s 23ms/step - loss: 4.7148 - mae: 1.6863 - val_loss: 4.7205 -
```

## 18. Evaluate All Models

```
models = {
    "MLP": (mlp_model, X_test_processed),
    "Residual MLP": (residual_mlp_model, X_test_processed),
    "Wide & Deep": (wide_deep_model, X_test_processed),
    "1D CNN": (cnn_model, X_test_seq),
    "GRU": (gru_model, X_test_seq)
}

results = []
predictions = {}

for model_name, (model, X_eval) in models.items():
    print("evaluating:", model_name)

    result, y_pred = evaluate_model(model_name, model, X_eval, y_test)

    results.append(result)
    predictions[model_name] = y_pred

results_df = pd.DataFrame(results).sort_values("RMSE").reset_index(drop=True)

results_df
```

```
evaluating: MLP
evaluating: Residual MLP
evaluating: Wide & Deep
evaluating: 1D CNN
evaluating: GRU
```

|   | Model        | MAE      | RMSE     | R2        | MAPE       |  |
|---|--------------|----------|----------|-----------|------------|---------------------------------------------------------------------------------------|
| 0 | Wide & Deep  | 0.493310 | 0.741112 | 0.832065  | 34.105637  |  |
| 1 | MLP          | 0.504357 | 0.755748 | 0.825367  | 36.886112  |                                                                                       |
| 2 | Residual MLP | 0.620030 | 0.883152 | 0.761524  | 37.183884  |                                                                                       |
| 3 | GRU          | 1.479540 | 2.031808 | -0.262231 | 159.886627 |                                                                                       |
| 4 | 1D CNN       | 2.087791 | 2.738388 | -1.292786 | 88.389565  |                                                                                       |

Next steps:

[Generate code with results\\_df](#)[New interactive sheet](#)

## ✓ 19. Actual vs Predicted Sample

```
best_model_name = results_df.iloc[0]["Model"]
best_predictions = predictions[best_model_name]

comparison_df = pd.DataFrame({
    "Actual Demand_kWh": y_test,
    "Predicted Demand_kWh": best_predictions
})
```

```
print("best model:", best_model_name)
```

```
comparison_df.head(20)
```

best model: Wide & Deep

|    | Actual Demand_kWh | Predicted Demand_kWh |
|----|-------------------|----------------------|
| 0  | 2.100             | 1.504218             |
| 1  | 0.480             | 0.758659             |
| 2  | 0.140             | 0.233807             |
| 3  | 2.980             | 2.699860             |
| 4  | 0.440             | 0.510668             |
| 5  | 2.980             | 2.884779             |
| 6  | 2.370             | 1.434441             |
| 7  | 1.990             | 1.727231             |
| 8  | 2.101             | 2.584913             |
| 9  | 0.700             | 1.182872             |
| 10 | 1.230             | 1.091517             |
| 11 | 1.070             | 0.975634             |
| 12 | 2.140             | 2.365438             |
| 13 | 1.130             | 1.025839             |
| 14 | 1.691             | 1.701282             |
| 15 | 2.060             | 1.696465             |
| 16 | 1.570             | 1.381047             |
| 17 | 0.197             | 0.393149             |
| 18 | 2.943             | 2.766057             |
| 19 | 0.380             | 0.610297             |



Next steps:

[Generate code with comparison\\_df](#)[New interactive sheet](#)

## 20. Actual vs Predicted Visualization

```
plt.figure(figsize=(14, 5))

plt.plot(
    comparison_df["Actual Demand_kWh"].iloc[:300].values,
    label="actual"
)

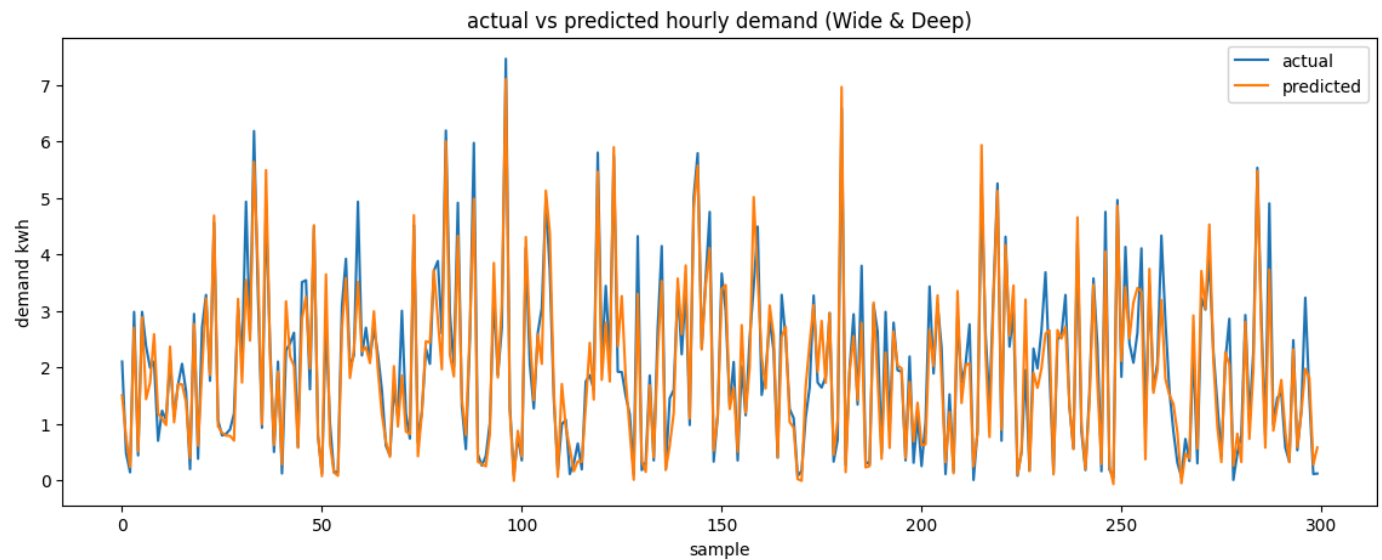
plt.plot(
    comparison_df["Predicted Demand_kWh"].iloc[:300].values,
```

```

        label="predicted"
    )

plt.xlabel("sample")
plt.ylabel("demand kwh")
plt.title(f"actual vs predicted hourly demand ({best_model_name})")
plt.legend()
plt.show()

```



## ✓ 21. Model Comparison Visualization

```

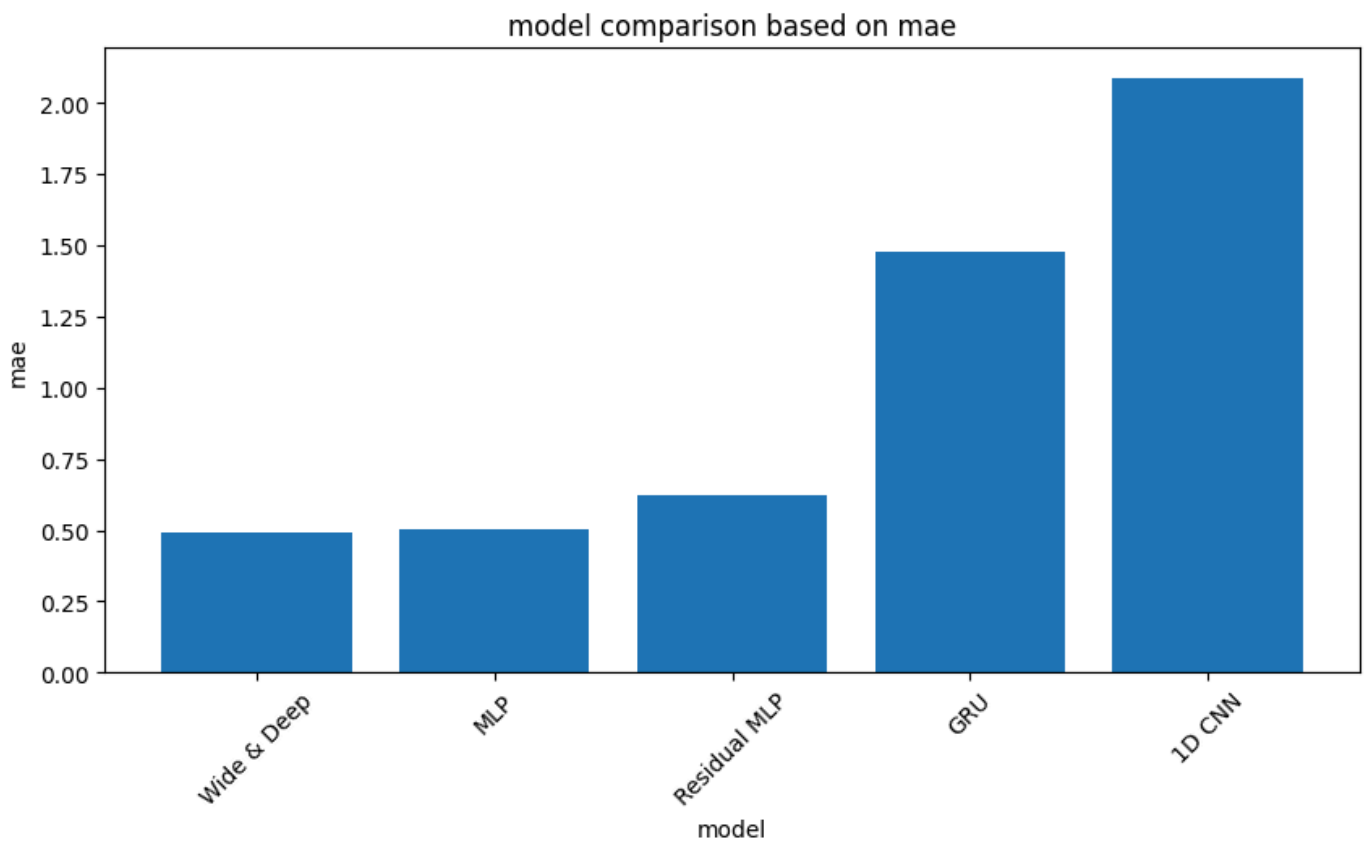
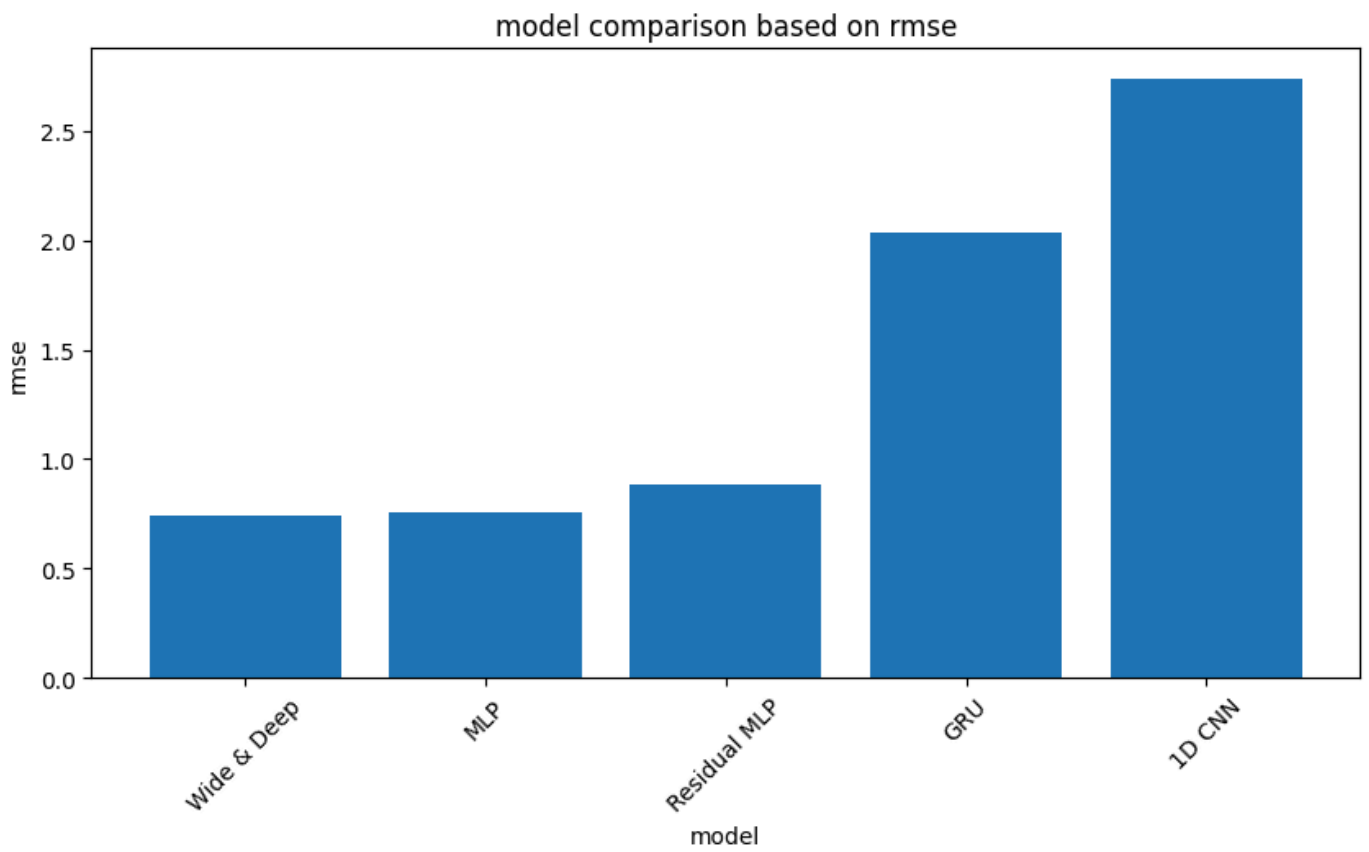
plt.figure(figsize=(10, 5))
plt.bar(results_df["Model"], results_df["RMSE"])
plt.xlabel("model")
plt.ylabel("rmse")
plt.title("model comparison based on rmse")
plt.xticks(rotation=45)
plt.show()

```

```

plt.figure(figsize=(10, 5))
plt.bar(results_df["Model"], results_df["MAE"])
plt.xlabel("model")
plt.ylabel("mae")
plt.title("model comparison based on mae")
plt.xticks(rotation=45)
plt.show()

```



## 22. Save Best Model

```

os.makedirs("react_artifacts", exist_ok=True)

best_model = models[best_model_name][0]

keras_model_path = "react_artifacts/best_wattwais_model.keras"

best_model.save(keras_model_path)

print("best model:", best_model_name)
print("saved keras model:", keras_model_path)

```

Best model: wide & Deep  
 saved keras model: react\_artifacts/best\_wattwais\_model.keras

## save preprocessing files

```

os.makedirs("react_artifacts/preprocessing", exist_ok=True)

joblib.dump(preprocessor, "react_artifacts/preprocessing/preprocessor.pkl")

scaler = preprocessor.named_transformers_["num"]

scaler_data = {
    "numeric_features": numeric_features,
    "mean": scaler.mean_.tolist(),
    "scale": scaler.scale_.tolist()
}

with open("react_artifacts/preprocessing/scaler.json", "w") as f:
    json.dump(scaler_data, f, indent=2)

```